SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – Case Study Analysis

Course Code:	CSA65	**Course Title:**	Generative AI and Large Language Models
CO Assessed:	CO2 – Precision (BL1, BL2)	**Assessment:**	Assessment Tool 2 – Case Study Analysis
Weightage:	20% of CIA	**Total Marks:**	100 (1 Question)
CO1 Statement:	*Apply prompt engineering techniques and utilize pre-trained Large Language Models through modern AI frameworks and APIs to solve real-world problems.(BL1, BL2)*		
Student Name:	C.Praveen Kumar	**Reg. No.:**	192472034
Section / Batch:	2024	**Date:**	01-08-2026

Code of Conduct

I C.Praveen Kumar *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____

CaseStudy: Embeddings for Semantic Search

Aim:

To develop a semantic search system using text embeddings that retrieves documents based on their meaning rather than exact keyword matching, improving search accuracy and user experience.

Introduction:

Traditional search engines depend on keyword matching. If users search with different words than those stored in the documents, relevant results may not be found.

Semantic Search solves this problem by understanding the meaning (context) of both the user's query and the stored documents.

This is achieved using Embeddings, where each sentence or document is converted into a numerical vector. Similar meanings produce similar vectors**.**

2. Problem Statement:

Design a semantic search application where users can search documents using natural language.

Example:

Documents:

- "Artificial Intelligence is changing healthcare."
- "Machine learning helps detect diseases."
- "Cricket is a popular sport."

User Query:

"How is AI used in medicine?"

Expected Result:

The first two documents should appear because they are semantically similar, even though the exact words are different.

3. Analysis:

Traditional keyword search has several problems:

- Cannot understand synonyms
- Cannot understand context
- Gives poor ranking
- Misses relevant documents

Semantic Search solves these issues by comparing the meaning of texts using embeddings.

4. Justification:

Semantic Search is useful in:

- Google Search
- ChatGPT document retrieval
- Amazon product search
- Hospital knowledge systems
- Legal document search
- University digital libraries
- Customer support chatbots

It improves user satisfaction by providing more meaningful results.

5. Suitable Model Selection:

Model	Purpose	Suitable?	Reason
GPT	Text generation	No	Generates text but not optimized for embeddings
BERT / Sentence-BERT	Generate embeddings	**Yes**	Best for semantic similarity
RLHF	Human feedback training	Not Required	Used for chatbot alignment
LoRA	Fine-tuning large models	Optional	If domain-specific improvement is needed
PEFT	Efficient fine-tuning	Optional	Reduces training cost

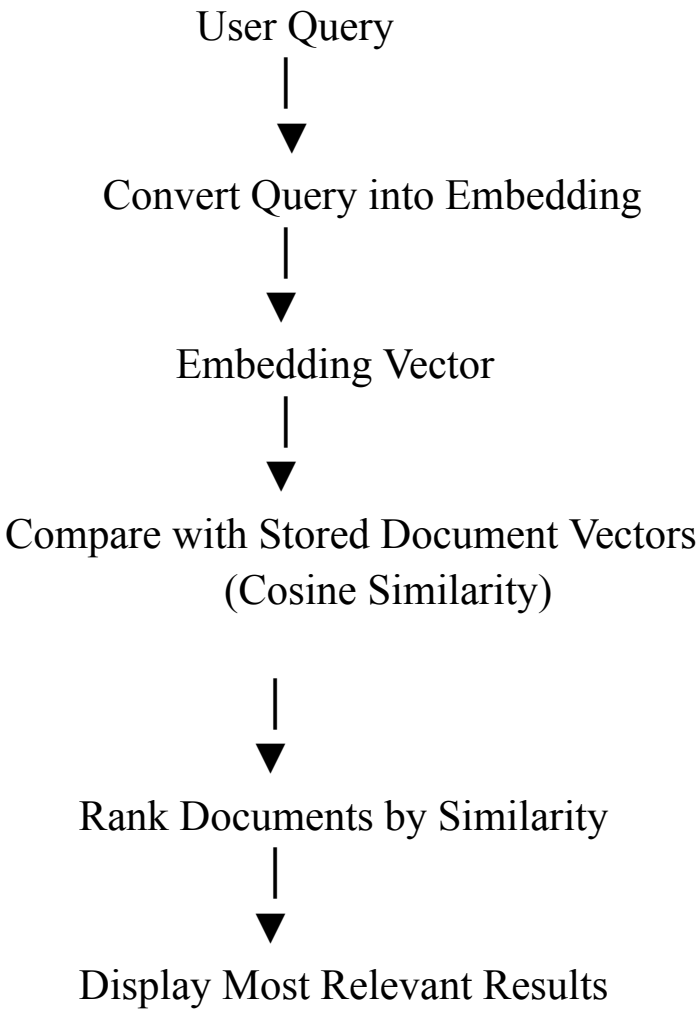
Selected Model

Sentence-BERT (SBERT)

Reason:

- Generates high-quality sentence embeddings
- Fast retrieval
- Excellent semantic similarity
- Widely used in industry

6.Architecture:



7. Architecture Explanation

Step 1

Collect documents.

Step 2

Convert every document into embeddings using Sentence-BERT.

Step 3

Store embeddings inside a vector database.

Step 4

User enters a query.

Step 5

Convert the query into embedding.

Step 6

Calculate cosine similarity between query embedding and document embeddings.

Step 7

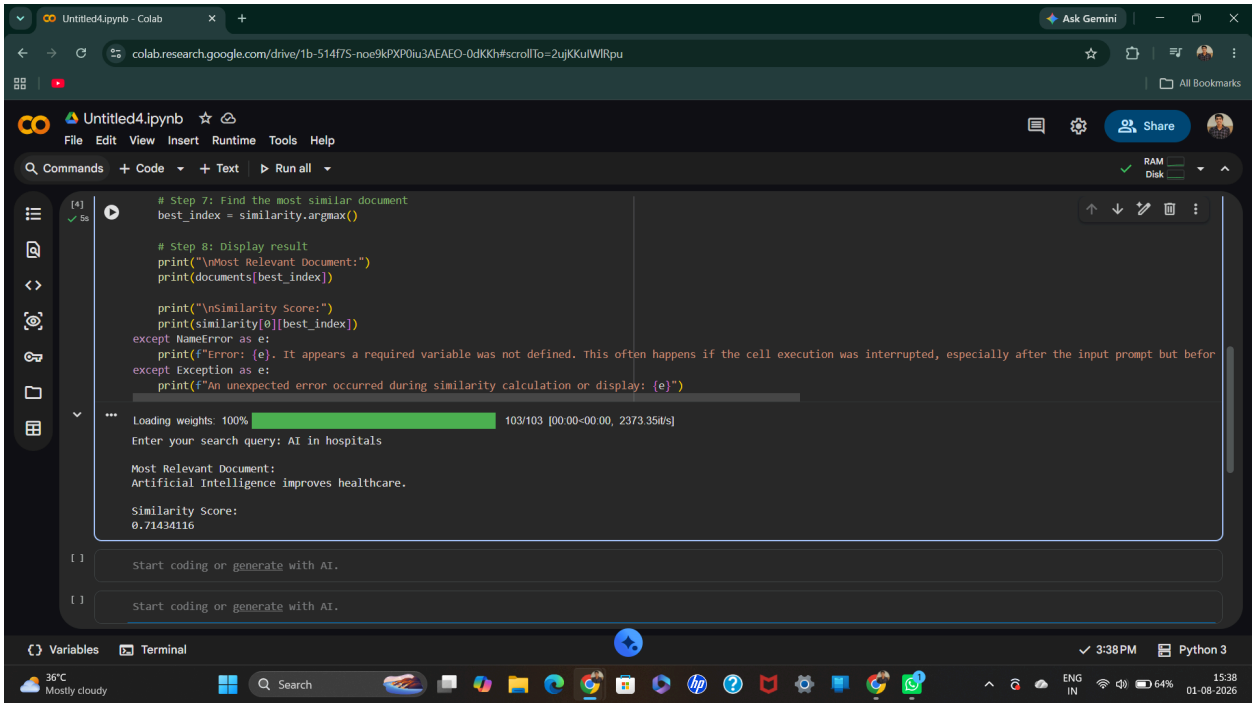
Return the highest-ranked documents.

8.Implementation Approach:

```
# Install first:
# pip install sentence-transformers scikit-learn

from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

# Step 1: Load the embedding model
model = SentenceTransformer("all-MiniLM-L6-v2")

# Step 2: Store documents
documents = [
    "Artificial Intelligence improves healthcare.",
    "Machine learning predicts diseases.",
    "Football is a popular sport.",
    "Deep learning is used in medical imaging."
]

# Step 3: Convert documents into embeddings
doc_embeddings = model.encode(documents)

# Step 4: Take user query
query = input("Enter your search query: ")

# Step 5: Convert query into embedding
query_embedding = model.encode([query])

# Step 6: Calculate cosine similarity
similarity = cosine_similarity(query_embedding, doc_embeddings)

# Step 7: Find the most similar document
best_index = similarity.argmax()
```

```
# Step 8: Display result
print("\nMost Relevant Document:")
print(documents[best_index])

print("\nSimilarity Score:")
print(similarity[0][best_index])
```

Output:



9. Suitable Python/API Approach

Python Libraries

- sentence-transformers
- transformers
- numpy
- scikit-learn
- faiss (for large datasets)

APIs

- OpenAI Embedding API
- Hugging Face Inference API
- Cohere Embedding API

For offline projects, Sentence-BERT from Hugging Face is sufficient.

10. Expected Output

Input:

AI used in hospitals

Output:

Artificial Intelligence improves healthcare

Although the words are different, the meaning is similar.

11. Advantages

- Understands sentence meaning
- Handles synonyms effectively
- Better search accuracy
- Faster retrieval using vector databases
- Supports multilingual search
- Improves recommendation systems
- Easy integration with chatbots and Retrieval-Augmented Generation (RAG)

12. Limitations

- Requires more memory than keyword search
- Embedding generation can be computationally expensive
- Performance depends on embedding model quality
- Domain-specific vocabulary may require fine-tuning
- Large datasets require vector databases such as FAISS, Pinecone, or Chroma

13. Evaluation

Evaluation metrics used for semantic search:

Metric	Description
Precision	Percentage of retrieved documents that are relevant
Recall	Percentage of relevant documents successfully retrieved
F1-Score	Balance between Precision and Recall
Cosine Similarity	Measures similarity between embeddings
Response Time	Time taken to return results

A good semantic search system should achieve:

- High Precision
- High Recall
- High Cosine Similarity
- Low Response Time

14. Real-World Applications

- Search engines
- ChatGPT Retrieval-Augmented Generation (RAG)
- E-commerce product search
- Legal document search
- Medical information retrieval
- Enterprise knowledge management
- Digital libraries
- Customer support systems

15. Conclusion

Embeddings-based semantic search is a modern approach that retrieves information based on meaning instead of exact keywords. By using **Sentence-BERT** to generate embeddings and **cosine similarity** to compare them, the system provides more accurate and context-aware search results. This approach significantly improves user experience and forms the foundation of many Generative AI applications, including Retrieval-Augmented Generation (RAG), intelligent chatbots, recommendation systems, and enterprise search solutions. While it requires additional computational resources and may need fine-tuning for specialized domains, its ability to understand semantic relationships makes it far superior to traditional keyword-based search systems.